LibRoad: Rapid, Online, and Accurate Detection of TPLs on Android

Jian Xu and Qianting Yuan

Abstract—Third-party library (TPL) detection plays a very important role in Android malware analysis. The focus of recent works has been shifted to the signature-based approach. However, previous methods have several limitations such as high time complexity and low precision, especially with the presence of similar TPLs and various versions of a TPL. To solve these issues, we propose a rapid, online, and accurate TPL detection approach, named LibRoad, which also follows the line of the signature-based research. To reduce the time cost, our approach integrates an application preprocessing component and a pairwise package matching component. The former divides an application into primary modules and non-primary modules to enable us to focus on analyzing packages in non-primary modules that are the most possibly imported from a TPL. The latter adopts a combination of the package name based matching policy for non-obfuscated packages and the signature-based matching policy for obfuscated packages, where the package name based matching policy has a lower time complexity than the signature based one. Further, to improve performance, our approach integrates a perfectly matched package and TPL determination component, which adopts the package filter mechanism, online TPL detection, and local TPL discovery to identify TPLs with low false positive and false negative. We conduct several groups of experiments on real-world applications and two ground truth bases. Experimental results show that compared to state-of-the-art approaches, LibRoad can achieve a high recall of 99.86 percent and a low false positive rate of 11.48 percent without the loss of efficiency.

Index Terms—Third-party libraries, obfuscation, library detection, similarity

1 Introduction

Third-Party libraries (TPLs), as a kind of important reusable software components, are gaining the attention of application developers and increasingly widely used in applications on mobile platforms. However, recent studies have shown that TPLs posed some threats including to track users for commercial interests by gathering location information and unique device identifiers [1], to infringe user privacy by collecting users' Email address or reading contact information [2], [3], [4], and to modify an original TPL by injecting malicious codes [5], on the privacy of mobile users. Moreover, TPLs also harm the accuracy of Android application analysis [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16] because it is difficult to differentiate the boundary of binary codes from between a TPL and user-developed codes. Hence, with the purposes of security protection and software analysis, there is a great need to develop a kind of TPL detection approach as a basis for subsequent Android application analysis tasks.

Reliably identifying libraries is a daunting but unresolved technical challenge. The main difficulties lie in two aspects. First, TPLs are tightly integrated into their host applications by implicitly linking them to applications' byte-codes, which leads to trouble in differentiating the boundary. Second, the adoption of the byte-code obfuscation techniques converts all identifiers into short and meaningless strings for the intellectual property protection and security guarding, which leads to trouble in identifying TPLs by package names and structural information. Thus, it deserves further research to provide a reliable and accurate TPL detection approach to address these challenges.

Recently, although the importance of TPL detection for Android application analysis tasks has been recognized, there is little attention paid to it. The existing work can be roughly categorized into three groups: the whitelist based method [17], [18], [19], [20], the machine learning-based method [21], [22], [23], [24], [25], and the signature-based method [26], [27]. The whitelist based method is simple, but fails to detect TPLs in case of obfuscation and has to continuously update the built whitelist of TPLs. The machine learning-based method mines shared code features in a large number of applications by applying clustering algorithms and identifies TPLs by matching shared code clusters. Such a method requires a large number of applications as input and is limited to detect TPLs that are not used by many applications. Moreover, this method fails to determine whether a specific TPL exists in a given application or not. The signature-based method detects TPLs used in an application by comparing a fuzzy elastic signature of the library and the application in a pairwise way. The representative approaches are LibScout [26] and LibPecker [27]. Compared with the former two methods, although the signature-based method works best in terms of accuracy, it also suffers from several drawbacks, specifically in case of the large-scale library detection. First, the pairwise comparison way that used in the signature-based method is inefficient. To discover all TPLs from a given application, they have to compare tens of thousands of TPLs from the library

• *The authors are with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China. E-mail: dolphin.xu@njust.edu.cn, qt1156962182@163.com.*

Manuscript received 11 Nov. 2019; revised 11 June 2020; accepted 15 June 2020. Date of publication 18 June 2020; date of current version 3 Dec. 2021. (Corresponding author: J. Xu.)
Digital Object Identifier no. 10.1109/TMC.2020.3003336

1536-1233 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

repository with the application. It is a very time-consuming process because many irrelevant application packages cannot be filtered out before comparison. Second, the existing approaches in the category of the signature-based method have a high false positive because they determined simply whether a library was used in an application by measuring the similarity between the TPL and the app. Such a comparison is less rigorous and responsible for a high false positive because it ignores the case that there are many similar lib packages among different libraries. Finally, the signature-based method is sensitive to the mutation of TPLs. Customization development is very common for developers by adding appropriate functions or pruning redundant functions on original TPLs, especially when these TPLs are not directly applicable to applications. However, even minor modifications to TPLs may result in false negatives.

To deal with these issues, we propose a rapid, online, and accurate approach for identifying TPLs incorporated in Android applications, named LibRoad. Although our approach partially follows the idea of the pairwise and signature based package comparison, we elaborately design several components to improve the efficiency and performance of TPL detection as greatly as possible. First, we integrate an application preprocessing component consisting of a root package analysis sub-component and a non-primary module and obfuscation identification sub-component, which aims at reducing the number of packages that need to be compared pairwise by identifying non-obfuscated packages and non-primary modules. Second, we integrate a package determination component consisting of two different package matching policies and a package filtering mechanism, which also aims at improving the efficiency of pairwise package comparison. Last, we integrate a TPL determination component consisting of two kinds of complementary TPL detection methods and a TPL discovery sub-component, which aims at reducing false positives and false negatives. To evaluate our approach, we conduct comprehensive experiments on real-world Android apps with a wide variety and two ground truth bases to demonstrate its performance and efficiency and compare with state-of-the-art approaches.

In summary, we make the following contributions in this paper:

1. We propose an Android application preprocessing approach to identify root packages and differentiate primary and non-primary modules, obfuscated and non-obfuscated packages, which can greatly improve the efficiency by reducing the number of packages that need to be analyzed.
2. We conduct a flexible pairwise package comparison by applying the package name based matching policy to non-obfuscated packages and the signature based matching policy to obfuscated packages respectively, which can also improve the efficiency because the former is faster than the latter.
3. We propose a filter-based perfect package determination approach by considering two kinds of extra information, partially obfuscated package names and package levels, which can reduce false positives.
4. We propose an accurate TPL determination approach by combining local repository oriented TPL detection, online detection, and TPL discovery, which can reduce false negatives. To the best of our knowledge, this is the first work to not only detect TPLs with the accurate version but also provide the capability of online detection.

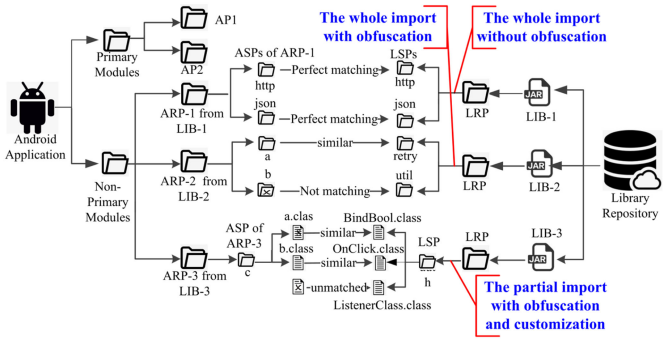


Fig. 1. The composition of a recompiled APK and the correspondence relationship with TPLs.

The rest of the paper is organized as follows. Section 2 describes the backgrounds that are helpful to understand our approach. Section 3 proposes a rapid, online, and accurate TPL detection approach. Experiments on a local repository and two ground truth bases are conducted and results are analyzed to demonstrate the effectiveness and efficiency of our approach in Section 4. Section 5 describes the related work in the same literature. Some relevant problems are discussed in Section 6. Finally, Section 7 concludes the paper.

2 Background

To provide some helpful design guides, we simply review the structure of an Android application and possible development behaviors related to TPLs, as shown in Fig. 1.

In general, an Android application consists of many application packages (APs), and these packages can be divided into two categories, primary modules and non-primary modules, where APs in primary modules are developed by developers to define application functionalities, while APs in non-primary modules are imported by developers to provide routines and utility libraries for primary modules, and each AP corresponds to a library package (LP) from a TPL. Moreover, some APs can also belong to an application root package (ARP) if they satisfy a specific constraint, and they become application sub-packages (ASPs) of this ARP. We will describe the terms ARP and ASP in the next Section by using an illustrative example. Each ARP corresponds to a TPL and owns one or multiple ASPs. Each TPL contains a library root package (LRP) and each LRP owns one or multiple library sub-packages (LSPs). Since TPLs are widely used in mobile applications, we simply review several kinds of development behaviors.

The first kind of behavior is to import a TPL into an application. There are three cases, including complete import, partial import, and customized import. For the complete import, all packages and classes in a TPL are wholly integrated into an application. For the partial import, only parts of packages and classes in a TPL are integrated into an application by developers. For the customized import, some packages in a TPL are modified by removing some classes

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

originally contained in these packages, or some classes in these packages may be modified by removing some members including methods and fields, and then the TPL is imported into an application. The last two cases will result in that the number of packages and classes from a specific TPL in an application is different from that of packages and classes in the original TPL, which is one of the main causes to lead to false negative for state-of-the-art TPL detection approaches. After importing some TPLs into an application, developers design and implement the application's functions by adding codes organized in a package way. It is not easy to differentiate them in a recompiled application. Only packages in the non-primary modules source from TPLs, which means that a TPL detection approach will have high efficiency to detect TPLs if the approach focuses on packages in non-primary modules.

The second kind of behavior is to obfuscate an application before releasing it. The identifier renaming is a kind of widely-used code obfuscation technology, which generally replaces identifier names such as package names, class names, and method names, with short and meaningless strings to make attackers difficult to understand them. Both application codes and identifiers from the imported TPLs may be obfuscated. The developers decide to choose packages that are obfuscated. Hence, for a recompiled application, we can find that some packages are obfuscated while the rest is not. Moreover, we also find that some package names are partially obfuscated. The main cause lies in that developers can choose the obfuscation scope in a fine-grained way. For example, package "com.karumi.dexter.listener" can be transformed into "com.a.b.c" by keeping the top-level package name unchanged. The most important impact of obfuscation on TPL detection approaches is to make the identifier name based matching not work for obfuscated packages.

The last kind of behavior is to optimize an application before releasing it. The main purpose of optimization is to make Android applications smaller and run faster. Using code optimization tools, some unused imported packages, classes, and methods may be automatically removed from an application during the optimization stage. A direct impact of optimization on TPL detection approaches is that the imported packages, classes, and members may have a great difference with their original ones, which means that the signature-based matching approach also cannot work well.

3 TPL Detection Approach

Modern software systems build on a significant number of TPLs to deliver feature-rich and high-quality software in a cost-efficient and timely manner. As a consequence, these systems contain a considerable amount of third-party libraries. We propose a rapid, online, and accurate detection approach, named LibRoad, to accurately discover the TPLs used in Android applications.

3.1 Framework

The idea behind our approach is to conduct the pairwise comparison between an application and a TPL by making full use of information extracted from the application and the TPL in order to determine whether the TPL is imported by the application. Fig. 2 describes our framework, which logically consists of four components including an application preprocessing component, a signature and weight generation component, a package determination component, and a TPL determination component.

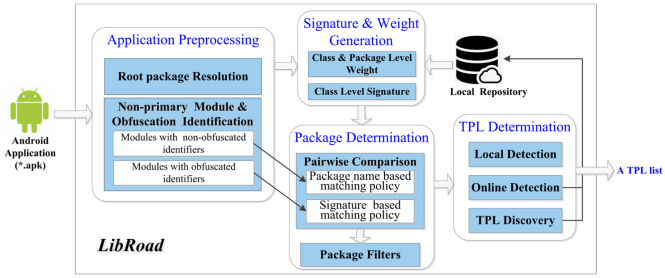


Fig. 2. The framework of LibRoad.

The application preprocessing component is used to decompose a given application, which includes three subcomponents. Using the root package resolution, we get all ARPs of the application. Using the non-primary module identification, we identify non-primary modules of the application because only packages in non-primary modules may source from TPLs. Further, we divide them into two subcategories, identifier obfuscated modules and identifier non-obfuscated modules, using an obfuscation identification method. Overall, this component aims at reducing the number of comparisons in the pairwise package comparison stage, which plays an important role in improving efficiency.

The second component is responsible for signature generation on packages and classes in a weighted way. This component can be applied to both applications and TPLs. Signatures and weights of all classes are generated and calculated respectively, followed by signatures and weights of all packages. These outcomes will be used by the package determination component.

The third component is responsible for finding a perfectly matched LP for a given ASP by a combination of package pairwise comparison and package filters. Here, the term "perfectly matched" denotes that the matching or determination algorithms used in this work attempt to identify the exact packages or TPLs from Android apps as accurately as possible. This component determines the matching policy depending on whether the ASP is obfuscated or not. Specifically, the package name-based matching policy is applied to a non-obfuscated ASP, while the signature-based matching policy is applied to an obfuscated one. For each ASP, we can find several LPs matched with it. Thus, a filter based perfect package determination algorithm is proposed to determine the perfectly matched LP for an ASP.

The last component aims at determining a perfectly matched TPL for an ARP. This component integrates a local repository oriented TPL detection, an online TPL detection, and a TPL discovery, to reduce false negatives.

Besides these four components, there is a local repository including real-world TPLs and Android applications. The local repository provides data support for evaluating our approach. To construct a repository of known libraries with diverse versions, we consider two sources, one from the maven central repository [30] and the other from the public GitHub repository. We choose the former as the data source

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

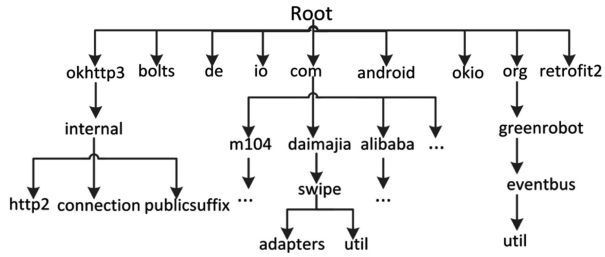


Fig. 3. The package structure of an example application.

to collect TPLs using the crawler technique because it provides a large volume of TPLs with diverse versions. For each TPL, we store auxiliary information such as binary codes, name, version number, and release date, which can usually be obtained by changing the log or directly from the host server. Currently, we have a repository of 12,000 TPLs and divide them into five groups: Advertising, Analytics, Cloud, Social Media, and Utilities, where Utilities account for the largest percentage of 60 percent in the repository because almost every Android application need use their functions. Second, to construct a repository of representative Android applications, we consider fifteen sources, including the official Google Play and fourteen third-party application markets, where f-droid (available at https://f-droid.org/) provides open-source projects. Totally, we collect 155,300 applications from these sources by filtering out redundant applications with the same hash value.

We will discuss the details of the components involved in the framework in the following texts.

3.2 Application Preprocessing

The application preprocessing component aims at improving the efficiency of our approach by adopting a combination of root package analysis, non-primary module identification, and obfuscation identification to reduce the number of comparisons and provide support for designing rapid pairwise package comparison methods.

3.2.1 Root Package Resolution

Both android applications and TPLs consist of multiple packages developed by Java programming language. Each package has a fully qualified name and each application can be represented as a package hierarchy tree. Different packages that share the same package name prefix constitute a root package. Take a real-world application "104_Job_Search_v1.11.0" for example, as shown in Fig. 3, if two packages "com.daimajia.swipe.adapters" and "com.daimajia.swipe.util" share the same prefix "com.daimajia.swipe", we say that package "com.daimajia.swipe" is an ARP, while packages "com.daimajia.swipe.adapters" and "com.daimajia.swipe.util" are ASPs.

To effectively identify all ARPs in an application, we propose a root package resolution algorithm using package hierarchy trees, as shown in Algorithm 1. This algorithm takes an installation file (.apk file) of a given Android application as inputs and outputs all ARPs and all sub-packages of each ARP. For each application, line 1 adopts reverse engineering technologies to get all application packages. Many widely-used reverse engineering technologies [28] can be used to accurately analyze and extract package names. After that, line 3 breaks each application package into several nodes based on the fully qualified package name, builds several package hierarchy trees, and gets all tree nodes. Lines 4 to 17 traverse each tree to find the longest path from the root node to a child node of the tree, where each node in the found path has at most one child. After getting all root packages, lines 18-26 find all sub-packages for each root package.

Applying the algorithm to the abovementioned example, we can get all ARPs. Obviously, the number of ARPs is less than that of APs. In the example application, the number of ARPs is 28, while the number of APs is 223.

Algorithm 1. Root Package Resolution Algorithm

Input: apk: an installation file of an Android application;
Output: *ARP*s: a set of root packages; <*arp*, *ASP*s>:a map of sub packages of each $arp \in ARPs$

```
1: APs = getAllPackagesofAPK(apk);
2: ARPs = new HashSet<>();
3: PackageTree nodes = getPackageTreeNodes(APs);
4: for (PackageTreeNode root: nodes) do
5:   childs = getChildNodes(root);
6:   if (isEmpty (childs)) then
7:     addRootPackage(root, ARPs);
8:   else
9:     for (PackageTreeNode child: childs) do
10:      getCurrentARPName(root,child);
11:      offsprings = getChildNodes(child);
12:      if (getFirst(offsprings))
13:        getCurrentARPName (“.”+ offsprings [1]);
14:      end if
15:    end for
16:  end if
17: end for
18: for each arp ∈ ARPs do
19:   ASPs = new HashSet<>();
20:   for each ap ∈ APs do
21:     if ap.isChildof(arp) then
22:       addSubPackage(ap, ASPs);
23:     end if
24:   end for
25:   contructPackageMap(arp, ASPs);
26: end for
27: return ARPs & {<arp, ASPs>};
```

3.2.2 Non-Primary Module and Obfuscation Identification

To further reduce the number of pairwise package comparisons, we combine a module decoupling approach and a package name obfuscation identification approach.

First, we present the module decoupling approach. The primary module can be easily identified using the metadata in the AndroidManifest.xml after reverse engineering. Specifically, the primary module can be inferred from the application package name declared in the AndroidManifest.xml that uniquely identifies an application on the device. Note that Android application development generally follows the package naming convention of Java programming language, where a developer generally uses the reverse Internet domain name to name a package to avoid conflicts with packages from other applications. Hence, we can find out the

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

```
<activity android:label="@string/AppName"
        android:name="com.m104.SplashScreenActivity">
  <intent-filter>
    <action android:name="android.intent.action.MAIN"/>
    <category android:name="android.intent.category.LAUNCHER"/>
  </intent-filter>
</activity>
```

Fig. 4. An example of primary module and its representative package.

application package name declared in the AndroidManifest. xml easily. Unlike applications running on computers, an Android application provides an entry activity to represent the entry function instead of the main function in C or Java. Take the application "104_Job_Search_v1.11.0" for example again, the metadata of an activity is given in its AndroidManifest.xml, as shown in Fig. 4, where it defines an activity named "com.m104.SplashScreenActivity" as the entry activity of the application with the action "android.intent.action. MAIN" as the entry point of the application. And the category "android.intent.category.LAUNCHER" means that the activity belongs to the loader class and can be loaded first once the application starts. As a result, we take the package with class "com.m104.SplashScreenActivity" as a primary package of the application. In this example, the ARP "com. m104" and all its ASPs are grouped together in the primary module. Compare with the other 27 ARPs, ARP "com.m104" is the one with the largest number of ASPs, removing it from the package list for comparison will be helpful for improving the detection efficiency.

After differentiating primary and non-primary modules, we present the package name obfuscation identification approach. According to [33], deciding whether a symbol name is obfuscated or not is a non-trivial task, which is out of the scope of this paper. Thus, we simply apply a single character-based parameter-free approach to detect package name obfuscation. Different from other kinds of identifiers, each segment of a package name is generally obfuscated by replacing it with a single character when the default settings of obfuscators are used. For example, the package "com.a" is regarded as an obfuscated one because the second segment of this package only contains a single character. Thus, if the default settings of obfuscators are used, our approach can work well. Of course, such a simple approach may suffer from false negatives and false positives if the customized settings of obfuscators are used. Some alternative approaches for identifier renaming identification can also be used here. For example, the authors in [34] proposed a heuristic approach based on word lists to detect the usage of identifier renaming. The detection accuracy of their approach depends on several factors, including the ways of word composition, the length of identifiers, and the threshold. Moreover, the setting of used obfuscators also has a significant impact on the detection accuracy. Thus, these alternative approaches can not only be more complex than ours but also suffer from false negatives and false positives. Using this approach, all ARPs can be further divided into two groups, obfuscated and non-obfuscated packages, denoted as P_o and P_{no} respectively.

3.3 Signature and Weight Generation

For obfuscated packages, to compare the similarity between two packages, we follow signature based comparison routines widely used in the field of program analysis. Previous approaches, such as LibScout and LibPecker, also use this way, and our signature and weight generation approach is similar to theirs. Here, we simply describe the signature and weight generation process, and the details can be found in LibPecker. Note that the signature and weight generation approach described here does not belong to our contribution of this paper.

Since classes are the most important parts responsible for specific functions for both a TPL and an application, we generate the class signature, class weight, and package weight, respectively. With the purpose of conveniently organizing classes and avoiding name conflicts, java package technique is used to organize classes into namespaces with a hierarchical name pattern. Each package has a few classes or sub-packages.

First, we generate a class signature for each class in a package by considering three factors: basic class information, methods, and fields. Specifically, for each class, we generate a basic signature, method signatures, and field signatures, respectively, compose them iteratively into a string, and generate a 32-bit hash value as the final class signature using an MD5 hash function.

Second, we generate class weights by considering their unequal contributions to a package. More weights need to be given to classes which make a package more significantly different from another. Intuitively, classes with more methods should have heavier weights, because functionalities of a package are implemented by these methods. Specifically, for a class c, its weight is calculated by Equation (1).

$$weight(c) = count(c.m) + deg^{+}(c) + deg^{-}(c), \tag{1}$$

where function count() denotes the number of methods, functions $deg^{+}()$ and $deg^{-}()$ count how many classes the class c depends and how many classes depending on the class c, respectively. The factors are robust against obfuscation techniques. Our insight is that although class names may be changed by obfuscators, dependencies are preserved. After assigning class weights, we normalize them into the interval between 0 and 1 by taking all class weights in the same package into consideration, denoted by $\overline{weight}(c)$.

Similarly, we assign weights to packages in a TPL or an ARP. The weight of a package p is calculated based on the number of classes in the package and the sum of weights of each class, as shown in Equation (2). After assigning package weights, we also normalize them into the interval between 0 and 1 by taking all package weights in the same TPL or ARP into consideration, denoted by $\overline{weight}(p)$.

$$weight(p) = count(p.c) + \sum_{i=1}^{n=|p|} weight(c_i). \tag{2}$$

3.4 Perfectly Matched Package Determination

In this section, we propose a perfectly matched package determination approach to find globally and perfectly matched LPs from the built local TPL repository for each ASP of a given Android application. Since packages kept after application preprocessing consist of non-obfuscated packages and obfuscated ones, we present two different package matching policies for them to reduce the time

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

expense of pairwise package comparisons, and they are the package name based matching policy for non-obfuscated packages and the signature based matching policy for obfuscated packages respectively. Further, we determine perfect matching LPs for a given ASP by applying package filters from the aspects of the package level and the partially obfuscated package name.

Formally, assume that the size of TPLs in the local TPL repository is q, L is a TPL of n packages from the repository, denoted as $L = \{lp_1, \ldots, lp_i, \ldots, lp_n\}$, and A is an application consisting of m_1 obfuscated sub packages and m_2 non-obfuscated sub packages in its non-primary modules, denoted as $ASP^o = asp^o_1, \ldots, asp^o_j, \ldots, asp^o_{m_1}\}$ and $ASP^{\bar{o}} = \{asp^{\bar{o}}_1, \ldots, asp^{\bar{o}}_j, \ldots, asp^{\bar{o}}_{m_2}\}$ respectively. The set of application packages in the non-primary modules is $A = ASP^o \cup ASP^{\bar{o}}$.

3.4.1 Package Name Based Matching Policy

Since packages of a TPL from the local TPL repository are not obfuscated, for each package $asp^{\bar{o}}_j \in ASP^{\bar{o}}$, we apply a package name based matching policy to find an accurate name matching LP from a TPL. This policy can work well due to the fact that Android application development follows the package naming conventions of Java programming language. More specifically, developers generally use the reverse Internet domain name to name a package to avoid conflicts with other applications or TPLs. It is rare for two packages developed by different providers to exactly have the same package name. Hence, such a policy provides an effective way to find an accurate matching LP.

For an arbitrary package pair $\langle asp^{\bar{o}}_j, lp_i\rangle$ from A and L, the similarity between them is calculated as Equation (3).

$$sim_{name}\left(asp^{\bar{o}}_j, lp_i\right) = \begin{cases} 1, & asp^{\bar{o}}_j.name = lp_i.name \\ 0, & otherwise \end{cases}. \tag{3}$$

Because there are no two packages with the same name in a TPL, once a LP is found to own the same package name with $asp^{\bar{o}}_j$, the search process terminates immediately. In the worst case, the time complexity of this policy to find an accurate package name matching LP from a local TPL repository for an ASP is $\mathrm{O}(q \times n)$. Here, we assume that the average number of LPs of the TPLs is n. Hence, the total time complexity is $\mathrm{O}(q \times n \times m_2)$.

3.4.2 Signature-Based Matching Policy

For obfuscated packages, we propose a signature-based matching policy to measure the matching degree between a LP and an ASP by iteratively comparing their package signatures, class signatures, members and fields signatures instead of directly comparing their package names. Due to the existence of customized development, it is unreliable to measure the similarity between a LP and an ASP from the aspects of package signatures and class signatures because packages and classes may be modified or removed. On the contrary, member signatures of a class in a TPL will be preserved in the application using the TPL. So, in our signature-based matching policy, we get the class similarity by measuring the similarity of member signatures, and then get the package similarity by measuring the weighted class similarity.

Given an Android application A and a TPL L, assume that *lp* is a library package owning z_1 classes in L and asp^o is an obfuscated package owning z_2 classes in A, denoted as $lp = \mathrm{c}^{\mathrm{lp}}_1, \ldots, \mathrm{c}^{\mathrm{lp}}_{\mathrm{i}}, \ldots, \mathrm{c}^{\mathrm{lp}}_{\mathrm{z}_1}\}$ and $\mathrm{asp^o} = \{\mathrm{c}^{\mathrm{asp}}_1, \ldots, \mathrm{c}^{\mathrm{asp}}_{\mathrm{j}}, \ldots, \mathrm{c}^{\mathrm{asp}}_{\mathrm{z}_2}\}$, respectively. First, the class similarity score between a library class $\mathrm{c}^{\mathrm{lp}}_{\mathrm{i}}$ and an application class $\mathrm{c}^{\mathrm{asp}}_{\mathrm{j}}$ can be measured by considering how many class members in $\mathrm{c}^{\mathrm{lp}}_{\mathrm{i}}$ are preserved in $\mathrm{c}^{\mathrm{asp}}_{\mathrm{j}}$, as shown in Equation (4). Second, for each application class $\mathrm{c}^{\mathrm{asp}}_{\mathrm{j}}$, we find the class with the highest similarity score from lp as its matched class, measure the similarity between an application class and a library package using Equation (5), and mark it matched. In the case that a large number of classes imported from *lp* are deleted, the calculation method of Equation (5) can avoid false negatives. Further, we calculate the package pair similarity by combing all class similarities in a weighed way, as shown in Equation (6). Compared with previous work [26], [27], our approach will no longer use a similarity threshold to determine whether a package pair is matched. Instead, we choose LPs with the highest similarity score for each asp^o_j as the best matched library packages after completing all pairwise package comparisons, even if the highest similarity score is close to 0. Obviously, compared with the package name based matching policy, the signature-based matching policy has a higher time complexity of $\mathrm{O}(q \times n \times z_1 \times z_2)$ to calculate the package pair similarity. Correspondingly, the total time complexity is $\mathrm{O}(q \times n \times z_1 \times \mathrm{z}_2 \times m_1)$.

$$sim_{c_to_c}\left(c^{asp}_j, c^{lp}_i\right) = \frac{\left|\left\{c^{lp}_i.member_signatures\right\} \cap \left\{c^{asp}_j.member_signatures\right\}\right|}{\left|\left\{c^{lp}_i.member_signatures\right\} \cup \left\{c^{asp}_j.member_signatures\right\}\right|} \tag{4}$$

$$sim_{c_to_p}\left(c^{asp}_j, lp\right) = \max\left\{sim_{c_to_c}\left(c^{asp}_j, c^{lp}_i\right)|^{|z_1|}_{i=1}\right\} \tag{5}$$

$$sim_{p_to_p}(asp^o, lp) = \frac{\sum_{j=1}^{|asp^o|} sim_{c_to_p}\left(c^{asp}_j, lp\right) \times \overline{weight}\left(c^{asp}_j\right)}{|asp^o|}. \tag{6}$$

3.4.3 Filter Based Perfect Package Determination

Using the package name based matching policy or the signature based matching policy, we may find that several matched LPs with the same and highest similarity score from different TPLs for a given ASP, and also find that some package names are not wholly obfuscated and their top level's package names are preserved, as shown in Tables 1 and 2. This case will result in false positives. As for the example in Table 1, the main cause that leads to such a false positive lies in that the pairwise package matching policy is a local one, and fails to check whether an ASP is paired with a sole LP or not. However, the example in Table 2 shows another different cause that leads to false positives, where package levels of a matched package pair are different.

To solve this issue, we propose a filter-based perfect package determination approach by considering two kinds of extra information, partially obfuscated package names and the number of package levels. Since java packages are named in a point-separated hierarchy pattern, we can roughly

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
An Example of the Partially Obfuscated Package Name Based Filter

ASP	matched LPs (Library\LP)	similarity score	Really Matched
io.a.a.a.a.d	fabric-1.4.4 \io.fabric.sdk.android.services.events	0.4842	√
	androidasync-1.3.7 \com.koushikdutta.async.http.socketio.transport	0.4842	×

TABLE 2
An Example of the Package Level Based Filter

ASP		matched LPs		similarity score	Really Matched
name	level	Library\LP	level		
com.b.a.d.b	5	glide-3.8.0 \com.bumptech.glide.load.engine	5	0.9545	√
		dexter-5.0.0 \com.karumi.dexter.listener	4	0.9545	×

derive information including package relationships (parent-child, sibling) and package levels from package names. Although the obfuscation of package names makes some information lost, occasionally available non-obfuscated partial package name information and always available package level information can be still be used to filter some matched LPs for a given ASP in order to reduce false positives. Hence, for a matched package pair $< asp^{o}, lp >$, if the number of package levels of asp^{o} differs from that of lp, the package lp cannot be a perfectly matched LP, and the package pair $< asp^{o}, lp >$ is removed. Further, if the top level package name of asp^{o} is non-obfuscated and differs from the top level package name of lp, the package lp cannot also be a perfectly matched LP. Similarly, the package pair $< asp^{o}, lp >$ also need be removed. According to these two filters, we can determine their perfectly matched LPs for the cases in Table 1 and Table 2 respectively.

3.5 Perfectly Matched TPL Determination

We propose a TPL determination approach consisting of the local repository oriented TPL determination, online determination, and the local repository update, where the first component aims at finding a matched TPL for an ARP, and the last two components ensure our approach without false negatives.

Given an ARP of application A, ARP $= \{\mathrm{asp_1}, \mathrm{asp_2}, \ldots, \mathrm{asp_i}, \ldots \mathrm{asp_m}\}$, we cannot always find perfectly matched packages for each ASP using the package name based matching policy or the signature-based matching policy. Specifically, for a non-obfuscated ASP in ARP, we may fail to find a perfectly matched LP with the same package name from the local TPL repository using the package name based matching policy because the local TPL repository is not always complete. In this case, the online determination is launched to avoid false negatives. Otherwise, the local repository oriented TPL determination process is conducted to determine a perfectly matched TPL for an ARP. To the best knowledge of ours, our approach is the first to provide the online determination and the local repository update. We will describe our approach in detail in the following texts.

3.5.1 Local Repository Oriented Determination

If all packages in an ARP have at least one matched LP, we apply the local repository oriented determination process to determine a perfectly matched TPL from the local repository. Compared with LibPecker [27], our approach searches the most similar LP for an ASP using Equation (7) instead of searching the most similar ASP for a LP using Equation (8). Further, we get the similarity score between an ARP and a TPL in a weighted way using Equation (9) instead of using Equation (10). Such a change makes our approach effectively avoid false positives, especially in the case that many imported classes are removed after compression and optimization, which mainly owes to the integration of the root package resolution in our approach. Besides this change, our approach applies a global searching policy to find a TPL with the highest similarity score as the perfectly matched one instead of relying on a threshold-based policy, which is also helpful to reduce false positives.

$$sim_{asp_to_l}\ (asp, l) = \max\{sim_{p_to_p}\big(asp, lp_j\big)|_{j=1}^{m}\ \} \quad (7)$$

$$sim_{lp_to_a}(lp, A) = \max\Big\{sim_{p_to_p}\big(lp, asp_j\big)|_{j=1}^{m}\Big\} \quad (8)$$

$$sim_{arp_to_l}\ (ARP, L) = \frac{\sum_{i=1}^{n} sim_{asp_to_l}(asp_i, L) \times \overline{weight}(asp_i)}{n} \quad (9)$$

$$sim_{l_to_a}\ (L, A) = \frac{\sum_{i=1}^{n} sim_{lp_to_a}(lp_i, A) \times \overline{weight}(lp_i)}{n}. \quad (10)$$

Take application "104_Job_Search_v1.11.0" for example, we describe the results of both our approach and the approach in the previous work [27], as shown in Table 3. In this example, the package "org.greenrobot.eventbus" is an ARP of the application from the TPL "eventbus-3.1.1", and owns two ASPs "org.greenrobot.eventbus" and "org.greenrobot.eventbus.util" with the weights 0.2182 and 0.7818, respectively. On the other hand, the TPL "eventbus-3.1.1" has three LPs "org.greenrobot.eventbus", "org.greenrobot.eventbus.util"and"org.greenrobot.eventbus.meta" with the weights 0.6553, 0.2969 and 0.04780, respectively. Obviously, using the approach in the previous work based on Equations (8) and (10), the similarity score is low, and close to 0, which makes us miss the right TPL. Instead, using our approach based on Equations (7) and (9), the similarity score is relatively high, and we consider that there is a higher probability to determine whether the package "org.

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
An Example of Old Strategy Causing False Negatives

ARP	ASP(weights)	matched LP for ASP (weight)	Similarity (previous)	Similarity (ours)
org.greenrobot.eventbus	org.greenrobot.eventbus(0.2182) org.greenrobot.eventbus.util(0.7818)	org.greenrobot.eventbus(0.6553) org.greenrobot.eventbus.util(0.2969)	0.0191 0.0365	0.9455 0.2432
-		org.greenrobot.eventbus.meta(0.04780)	-	-
Total Similarity Score			0.0234	**0.6918**

greenrobot.eventbus" was imported from the TPL "eventbus-3.1.1" or not.

3.5.2 Online Determination

The local TPL repository can never be complete for several reasons. First, resources assigned for the local TPL repository are always limited. Second, the local TPL repository is built using an imperfect collection policy. Last, new TPLs are developed continuously. Thus, we fail to determine whether an ARP is imported from a specific TPL or not, although we can determine that the ARP was not from the local repository. If the ARP is exactly from a specific TPL not contained in the local TPL repository, it will result in a false negative. To solve this issue, we integrate an online TPL determination component in our detection approach to mitigate its impact. To the best knowledge of ours, our approach is the first one with the capability of online determination.

The idea behind our online TPL determination is simple, and it searches possible TPLs from the maven central available from the website https://search.maven.org based on the straightforward package name matching of the ARP. If the search results are not empty, we download these matched TPLs (several versions of a TPL), store them into the local repository, and measure the similarity between the ARP and any version of the TPL. Otherwise, we consider that the ARP is not from a TPL. Specifically, once downloading completely when the search results are not empty, we first convert those files with the surfix 'aar' into dex files, and then apply the proposed approach to measure the similarity between the ARP and each version of the TPL, finally choose the version with the highest similarity score as the perfectly matched TPL. Thus, our approach can not only determine whether an ARP is from a TPL or not from a view of the Internet, but also determine the accurate version of a TPL. Of course, note that our online TPL determination only works on non-obfuscated ARPs. Fortunately, there is a relatively high percent of non-obfuscated ARPs in released applications according to the statistic results [25]. Hence, compared with previous work, our approach is helpful to reduce false negatives.

3.5.3 TPL Discovery From the Local Repository

Besides the incomplete local TPL repository, there is another case responsible for false negatives. Although some unknown TPLs were exactly integrated into Android applications, they could not be downloaded from the Maven repository, which will result in false negatives using all existing TPL detection approaches in the literature. To solve this issue, we integrate a TPL discovery component in our detection approach to mitigate its impact. The idea behind our TPL discovery is inspired by the clustering approach used in LibRadar [29]. The machine learning based method mines shared code features in a large number of applications by applying clustering algorithms, and identifies TPLs by matched shared code clusters. Such a method discovers TPLs if there is a similar set of components in different applications by comparing their signatures. Our TPL discovery component follows the same principle, and aims at exploring unknown TPLs that are pervasively used by many apps. To the best knowledge of ours, our approach is the first one with the capacity of TPL discovery and repository update.

4 Evaluation

In this section, we conduct comprehensive experiments on real Android applications and Android TPLs to evaluate LibRoad from the aspects of performance and efficiency. Meanwhile, we conduct a deep comparison with state-of-the-art approaches to demonstrate the advantages of our approach. Moreover, we present some important findings got from the evaluation process.

4.1 Ground Truth Base

To the best of our knowledge, there is no ground truth base about the application-library pairs used in Android for evaluating the performance of TPL detection approaches. To evaluate the performance of efficiency of LibRoad, we propose two methods to build two ground truth bases, respectively.

The first method is a simple and automatical one based on the following steps. First, we choose 300 applications and 6000 TPLs from the local repository and automatically decompile them with the tool apktool [28] to generate a group of smali files for each application. Second, for each application, we empty all TPLs used in the application by removing all files in the directory. Third, we randomly add a list of TPLs with a size of 8 to 10 for each application. And then, to make the ground truth construction more realistic, we modify those injected TPLs to a slight degree according to the following four policies: to keep the whole TPL without any change, to modify a few package names of the TPL, to obfuscate identifiers, and to delete a few packages, classes, and methods in the TPL. Finally, we inject a list of TPLs with or without changes into an application iteratively and repackage the application. To this end, we successfully construct a ground truth base of 2,800 application-library pairs, and name it GTB-1. On average, each application contains 9.33 TPLs.

The second method follows the same way used in previos work [25], [27] with a light change. We randomly choose 1000 real-world applications from the local repository to construct the ground truth. Specifically, for 500 apps

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
Evaluating the Contribution of Pluggable Components to the Performance and Efficiency of the Proposed Approach

Ground Truth	Component	Detected Pairs	True Positive	Recall	False positive ratio	Precision	Avg.Time per APK
2800 pairs (GTB-1) Customized	Primary module identification				-		
	Package name based comparison	2964	2796	99.86%	5.67%	94.33%	243.79
	Package Filters	4127	2796	99.86%	32.25%	67.75%	28.14
	Online Detection				-		
	The whole approach	2964	2796	99.86%	-	94.33%	30.63
14233 pairs (GTB-2) Real-world	Primary module identification	16063	14233	100%	11.39%	-	140.05
	Package name based comparison	16060	14233	100%	11.38%		791.42
	Package Filters	39969	14233	100%	64.39%		84.57
	Online Detection	7954	6134	43.08%	22.88%		56.66
	The whole approach	16060	14233	100%	11.38%		91.38

including source code from f-driod, we analyze their lib dependency configuraton file named 'build.gradle' to extract application-library pairs. For the remaining apps, we manually investigate them to get the groud truth like LibD and LibPecker. We acquire in total 14233 application-library pairs, and name it GTB-2. Compard to the first method, the second is more difficult and costs more time but more convincing.

4.2 Settings

We implement our approach *LibRoad* in Java (available at github https://github.com/njustbdag/LibRoad) and conduct extensive experiments on the local repository and two ground truth bases to evaluate the performance and the efficiency of our approach and state-of-the-art approaches. As for the performance, we consider four metrics, recall, precision, false positive rate (FPR) and false negative rate (FNR), as shown in Equations (11)-(14), respectively.

$$\text{recall} = \frac{\#of\ true\ positives}{\#\ of\ pairs\ in\ the\ ground\ truth} \tag{11}$$

$$\text{precision} = \frac{\#\ of\ true\ positives}{\#\ of\ detected\ pairs} \tag{12}$$

$$\text{FPR} = \frac{\#\ of\ detected\ pairs\ -\ \#\ of\ true\ positives}{\#\ of\ detected\ pairs} \tag{13}$$

$$\text{FNR} = \frac{\#\ of\ pairs\ in\ ground\ truth\ -\ \#\ of\ true\ positives}{\#\ of\ pairs\ in\ ground\ truth}. \tag{14}$$

As for efficiency, we consider several metrics related to time, including the application preprocessing time (denoted as t_1), the package name based matching time (denoted as t_2), the signature based matching time (denoted as t_3), the perfect TPL determination time (denoted as t_4), and the total time (denoted as T). The total time is calculated using Equation (15).

$$T = \sum_{i=1}^{4} t_i. \tag{15}$$

4.3 Evaluating Pluggable Components

We conduct the first group of experiments on two ground truth bases to evaluate the effectiveness of each individual component of the newly proposed detection framework. In our framework, four components or sub-components including primary module identification, package name based pairwise comparison, package filter, and online detection, are pluggable and specially designed to improve performance or efficiency. Other components or sub-components are necessary and cannot be removed. Hence, we evaluate these pluggable components in experiments by removing them from our framework one by one. The experiment results related to the performance in terms of recall and precision and the efficiency in terms of the average time per APK are shown in Table 4. Note that for customized applications, because they don't include primary modules and all TPLs are imported from the local repository, we needn't evaluate the primary module identification sub-component and the online detection sub-component. For real-world applications, we evaluate the false positive ratio instead of precision because we don't have the whole true positive application-library pairs. From Table 4, we get some findings on these components as follows.

First, the primary module identification component has a positive impact on efficiency but no impact on performance, which can help us save 34.76 percent detection time for each application from 140.05 to 91.38 seconds. We can conclude that this component is effective in improving the efficiency of our approach. Further, to discover the hidden reason, we analyze the distribution of the number of packages in primary modules on 300 real-world applications. Table 5 presents the distribution. About 60 percent of applications contain more than 20 packages in their primary modules. Obviously, excluding these packages from the pairwise package comparison stage can greatly improve efficiency, which strongly shows the importance of the non-primary module identification in the application preprocessing component.

TABLE 5
The Distribution of the Number of Packages That can be Excluded by Module Decoupling

# of packages in primary modules	# of applications
>5	273
>10	250
>15	199
>20	178

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 6
Obfuscation Results on Selected 300 Real-World Apps

	Applications		ARPs	
	OF*	NOF*	OF*	NOF*
#	118	182	709	6079
ratio	39.33%	60.67%	10.44%	89.56%

*Note: OF means Obfuscated;NOF means non-obfuscated.

Second, the package name based pairwise comparison component also has a great impact on efficiency but no impact on performance, which can help us save 87.43 and 88.45 percent detection time for each application from GTB-1 and GTB-2, respectively. This indicates that it is reasonable to integrate the package name based matching policy in our detection framework because real-world applications exactly contain many non-obfuscated packages and the package name based matching policy is faster than the signature-based one. The improvement space in efficiency depends on the number of non-obfuscated packages and obfuscated packages in a given application. We further analyze 300 real-world applications to find the distribution of the number of non-obfuscated packages and obfuscated packages, as shown in Table 6. We find that about 60 percent applications and about 90 percent ARPs are not be obfuscated respectively. According to the theoretical analysis, the time complexity of the package name based matching policy is less than that of the signature based matching policy. Hence, we can conclude that a combination of differentiating non-obfuscated packages and obfuscated packages and applying different package matching policies has a great contribution to reducing the detection time, which shows this component is effective.

Third, the package filter component has a great impact on both performance and efficiency. Specifically, this component can help us greatly reduce false positive ratio from 64.39 to 11.38 percent on GTB-1. From Table 5, we can see that the detected pairs significantly increase on two ground truth bases and the detected true positive pairs remain unchanged when the pair filter component is removed. This indicates that integrating this component into our framework is helpful to reduce false positives. Of course, we also observe its negative impact on efficiency. The average detection time for each application increases by 8.85 and 8.05 percent on GTB-1 and GTB-2, respectively.

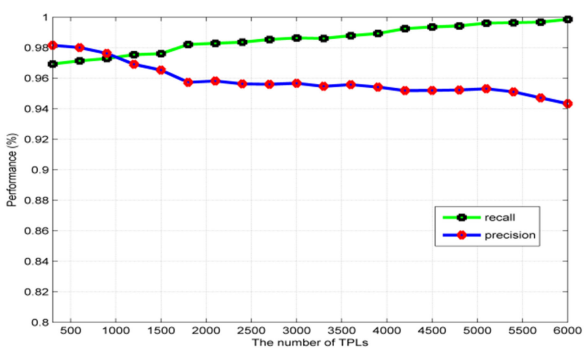


Fig. 5 The results of varying TPLs.

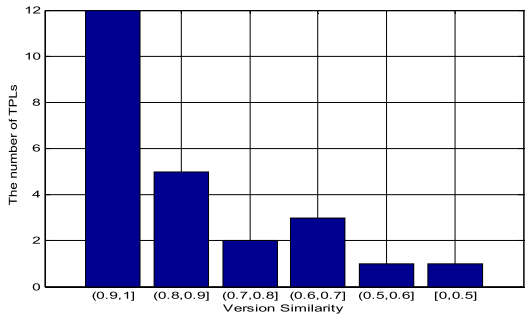


Fig. 6. The distribution of version similarity for TPLs appeared in the false negative application-library pairs.

Last, the online detection component also has a great impact on both performance and efficiency. Specifically, this component can help us greatly improve recall from 43.08 to 100 percent and discover more pairs from 7,954 to 16,060 pairs. This indicates that online detection provides a powerful capacity to make our detection more robust because the pre-built local TPL repository is always incomplete. To further demonstrate its performance in the case of an incomplete TPL repository, we conduct another experiment on GTB-1 and an incomplete TPL repository. Such a setting accords with the actual situation and will activate the online TPL detection. In our experiment, we vary the number of TPLs in a new TPL repository from 300 to 6000 with a step 300 through randomly choosing TPLs from the local TPL repository. Fig. 5 shows the results of both approaches. We can see that LibRoad is not sensitive to the varying of the number of TPLs. As continuously increasing the number of TPLs, although LibRoad has an increasing recall and a decreasing precision, it performs stable, which shows that LibRoad can tally with the real TPL detection situation of no complete TPL repository. On the other hand, similar to the package filter component, the online detection component imposes a negative impact on efficiency. Specifically, the average detection time for each application increases 61.28 percent on GTB-2, which depends on multiple factors including the size of the downloaded TPL, network time delay, and the processing time.

4.4 Evaluating the Whole Approach

We also evaluate both performance and efficiency of the whole detection approach. From Table 4, we can find that our approach reaches to a precision of 94.33 percent and a recall of 99.86 percent on GTB-1 and a false positive ratio of 11.38 percent and a recall of 100 percent on GTB-2, respectively. Further, we also observe that our approach also suffers from 168 and 1827 false positive pairs on GTB-1 and GTB-2, respectively. Taking the former as an example, we perform further analysis to figure out the causes. We find that 24 TPLs appear in 168 false positive application-library pairs, owning several versions in the local TPL repository. And, by computing the similarity of different versions of each TPL, we find that different versions of a TPL have high similarity. Fig. 6 presents the distribution of version similarity. We can see that the percentage of TPLs whose version similarity scores exceed 0.7 reaches to 79.2 percent, which indicates that although our approach can determine the concrete version of a TPL, some extra studies are still needed to reduce false negatives from similar versions of a TPL.

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 7
Comparison With Three Known State-of-the-Art Approaches on GTB-2 (14233 Pairs)

	# of detected pairs	# of true positives	FPR (%)	FNR (%)	Time (sec.)
LibRadar	9990	5481	45.13	61.67	17.8
LibD	12107	9592	20.77	32.61	20.5
LibPecker	19392	13872	28.47	2.54	602.2
LibRoad	16060	**14233**	**11.38**	0	91.4

Besides false positives, we also observe that our approach fails to detect some application-library pairs on GTB-1. To discover their causes, we conduct a manual examination to figure out why our approach does not flag them as positive pairs. By deep analysis, we find that the reason for false negative is that the TPLs incorporated in applications significantly differ with their original ones, which means that our approach cannot effectively detect those TPLs extremely modified, and fails to measure the degree of TPL modifications in a quantitative way. But we argue that false negatives of our approach are acceptable because as a matter of fact, extreme modification on a TPL leads to a new one from aspects of package relationship, the number of packages or classes, etc.

4.5 Comparison With State-of-the-Art Approaches

We compare our approach, LibRoad, with three known state-of-the-art approaches including LibRadar [29], LibD [25], and LibPecker [27] on the ground truth from real-world applications. Although these tools are available at Github, Android apps and third-party libraries used in their experiments are unavailable. Hence, we evaluate all approaches on both apps from GTB-2 and the same local TPL repository containing more than 12,000 TPLs, whose size is close to the number of libraries identified by LibD and LibRadar on more than 1 million apps. For each TPL, its feature hash values are generated according to the feature hashing component used in LibD and LibRadar or the package signature component used in LibPecker, respectively. Given a testing app, instances of its potential libraries as well as features of each instance are generated, and its feature hash value is compared with those of TPLs one by one. If feature hash values are exactly equivalent, a TPL is identified.

Table 7 presents the performance of all approaches. From Table 7, we can see that LibRoad identifies all true positive pairs on GTB-2, while other approaches miss some. The possible cause lies in that the libraries might be absent in the local TPL repository even the size of the repository is large. The online detection can effectively solve this issue. Moreover, although all approaches suffer from false positives, LibRoad performs best among them. A possible cause is that our approach takes the library customization and obfuscation behaviors into accounts by integrating a more fine-grained package and library matching component in our framework instead of enforcing strict comparison between two package feature hash values. LibPecker adopts a similar matching policy with ours. Hence, its FNR is also lower than LibD and LibRadar. Of course, we think that LibRadar and LibD probably have better performance on large datasets since they apply the clustering-based TPL identification method.

Further, we evaluate the efficiency of all approaches in terms of the detection time. For LibRadar and LibD, the detection time mainly includes time to get an intermediate representation, time to recover instances of packages, time to generate package feature hash values, time to enforce pairwise package feature comparison. For LibPecker, it is similar to LibRoad without the primary module identification time and the online detection time. The last column of Table 7 presents the detection time of all approaches. We can see that LibRoad and LibPecker need more time than LibRadar and LibD, because LibRoad and LibPecker integrate a more fine-grained comparison policy. Moreover, LibRoad performs better than LibPecker in terms of the detection time by integrating the primary module identification sub-component to reduce the number of pairwise comparisons and applying the package name-based comparison policy for non-obfuscated packages. Overall, LibRoad is quite efficient.

5 Related Work

Recently, many research efforts have been taken to detect TPLs in Android applications. Existing studies can be roughly categorized into three groups: the whitelist based approach [6], [17], [18], [19], [20], the machine learning based approach [21], [22], [23], [24], [25], [29], and the signature-based approach [26], [27].

The whitelist based approach manually creates and continuously maintains a whitelist of known TPLs, and performs pairwise comparisons with a given Android application using the name or the hash value of a TPL. For example, Chen et al. [6] use a whitelist of 73 TPLs to remove common libraries from their application clone analysis. AdRisk *et al.* [18] suggest identifying the potential risks of the ad library by using a whitelist of 100 ad libraries. However, the whitelist based approach has three main drawbacks. First, it cannot deal with the situation of applying obfuscation techniques such as identifier renaming. Second, it needs to continuously maintain the white list to keep up to date. Third, it can only detect TPLs that appeared in the whitelist.

To deal with obfuscation, a kind of more robust approaches based on machine learning has been studied, which generally extracts library-level features, or module-level features, or package-level features, and applies machine learning algorithms on feature vectors to detect TPLs. AdDetect [22] is an automated advertising library detection framework based on semantic analysis on module-level features using SVMs. PEDAL [23] also detects advertising libraries using the classifier trained on feature vectors extracted from SDK library codes and package relationship information. Compared with the whitelist based approach, an advantage of the machine learning based approach is that it can work when the detected application is obfuscated. But, a limitation of this approach is that it does not work if developers modify the codes of imported TPLs. The last is the signature or profile-based approach, which identifies incorporated TPLs from applications by comparing signatures between a reference library and an application in a pairwise way. LibRadar [29] uses hash-based profiles to detect similar libraries, where profiles are generated from the frequency of API calls within

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

a distinct package. LibD [25] divides an application into several components as library candidates, extracts stable features that are obfuscation resilient in most cases from candidate components, generates signatures using a feature-based hash algorithm, and detects libraries if there is a similar set of candidate components in different applications by comparing signatures. The strength of their approach is that it can detect unknown libraries. But these approaches also come with several drawbacks, they rely on such a basis that libraries are pervasively used by many apps. Once TPLs are rarely used, they cannot be detected accurately. Also, the features used in these approaches such as API calls for clustering are likely not obfuscation-resilient enough in most cases. For example, DexGuard [31] can obfuscate libraries with API hiding or control-flow randomization. Moreover, facing the library customization practice, these approaches also suffer from false positives and false negatives. The first work to deal with the abovementioned issues is LibScout [26]. It constructs obfuscation-resilient class profiles for both original TPLs and applications and detects TPLs by comparing the similarity between a TPL and an application according to their profiles. Their approach comes at the cost of completeness but has several advantages such as fewer false positives and the possibility of inferring exact library versions. However, through a deep study of library integration practice, we find LibScout is limited in its adaptability to deal with the case of library customization and elimination. A recent work, LibPecker [27], extends LibScout by exploiting the internal dependencies inside a library to profile a library and assigning weights to library classes according to their contributions to the library. Compared to LibScout, their approach improves recall and precision. Although the results reported by LibScout and LibPecker have been very promising, they are still not as good as they could have been due to the limitation of efficiency. PANGUARD [32] provides a promising solution to improve detection efficiency by introducing an edge computing-assistant TPL detection architecture as well as an accurate TPL detection approach. Compared to PANGUARD, our solution still focuses on the TPL detection methodology and designs several key components to improve efficiency instead of using extra assistant techniques like edge computing.

To further improve the performance and efficiency of TPL detection, we propose a novel approach that can effectively overcome the above limitations. Compared to previous work, the difference mainly lies in the following. To improve efficiency, we identify non-primary and obfuscated modules of applications by considering various relationships to reduce the size of pairwise comparisons. To improve performance, we integrate critical components including a filter-based package determination, online detection, and TPL discovery, to optimize the detection process and overcome the limitation of incomplete TPL repository. Moreover, we construct a ground truth base for colleagues to evaluate the detection performance.

6 Discussion

Although our approach has demonstrated promising potentials in efficiently and accurately detecting TPLs, we still need to discuss some limitations and open problems for possible further improvements as well as additional findings.

TABLE 8
The number of TPLs used in applications

# of TPLs in an application	# of applications	Percent (%)
>20	408	40.8
(15, 20]	209	20.9
(10, 15]	171	17.1
(5, 10]	113	11.3
(0, 5]	99	9.9

6.1 Limitations

The first limitation is that our approach fails to detect TPLs that are significantly modified, although it can tolerate a slight TPL customization and code elimination. When most codes in a TPL are modified during the development process, it becomes a new TPL with completely different behaviors in fact. Once these significantly modified packages are obfuscated, the online detection component does not work, and we have to only rely on the local detection using the local TPL repository. Inevitably, our approach will suffer from false negatives. In our future work, we plan to solve this issue by deep analyzing significantly modified TPLs and improving the TPL determination algorithm. The second limitation is that our approach still has a high time cost of about 90s per application, although the time cost is significantly lower than LibPecker. Since our approach still partially follows the signature based pairwise package matching policy on obfuscated packages, the signature based comparison time contributes most to the total time. Hence, thoroughly abandoning the signature based pairwise package matching policy seems inevitable. We are trying to develop a package structure information based TPL detection approach as an alternative approach to replace the signature based one. Although we have preliminary results on this direction, to perfectly solve this issue, it will still deserve further study.

6.2 Additional Findings

We have some additional findings about the TPL detection and the usage of TPLs.

The first finding relates to the popularity and usage of TPLs, as shown in Tables 8 and 9, respectively. Table 8 shows the number of TPLs used by 1,000 applications. The statistical results show that all applications use TPLs, and about 78.8 percent of applications use more than 10 TPLs, which reflects the proliferation of TPLs in Android applications, and also indicates the importance of the TPL detection in Android application analysis. Table 9 lists the most popular top 10 TPLs excluding the Android support libraries according to the frequency of the identified TPLs. Most of them belong to the category of utilities, which indicates that Android application developers generally introduce these TPLs to help them develop and strengthen application functions.

The second finding relates to obfuscation and customization of TPLs. We find that about 40 percent of the identified TPLs are incorporated in applications without any modification and the identifiers of only about 18 percent of the identified TPLs are obfuscated, which can provide a guide in improving efficiency by adopting the package name matching. We also find that most TPLs that are not obfuscated and

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

TABLE 9
Top 10 Most Popular Android TPLs

Most popular TPLs	#	Category
facebook	836	Social media
Bolts	808	Utilities
Facebook-Audience	691	Advertising
Umeng.Analytics	676	Analytics
Okio	647	Utilities
Okhttp3	622	Utilities
Apache.Commons	576	Utilities
Kotlin	500	Utilities
Zxing	484	Utilities
Flurry.Analytics	452	Analytics

customized are widely-used Android development tool libraries, while ad libraries, analytics libraries, and social media libraries are more likely to be obfuscated and customized. This finding will be helpful to understand the intent of Android application developers in incorporating TPLs. The intent that ad libraries, analytics libraries, social media libraries, etc., are obfuscated and customized may lie in that application developers try to avoid applications being analyzed and repackaged for malicious purposes.

6.3 Non-public TPLs

In our work, we have observed that some TPLs discovered from apps are not available on public repositories because they may be distributed through private channels or only be used by apps developed by the same company. Such a case can result in false negatives. Hence, we introduce a TPL discovery component to mitigate its impact. The idea behind our TPL discovery is inspired by the clustering-based approach used in LibRadar and LibD. We have discovered 14 non-public TPLs in our experiments. For example, package "io.card.payment" is found to be shared by more than 20 apps such as "85_Cafe_v1.0.8_apkpure" and "Angel_Taxi_v4.2.5_apkpure.com.apk" in our app repository.

However, the number of discovered non-public TPLs is small, which depends on the size of the dataset. To better deal with this case, it is necessary to further expand the size of the app repository.

7 Conclusions

To deal with the issues of low efficiency, high false positive and false negative in the signature-based detection approaches, we propose a rapid, online, and accurate detection approach. Some observations from the real situation of TPL usage provide useful insights to design our approach. The root package resolution is used to divide application packages into several ARPs, and each ARP corresponds to at most one TPL, which reduces false positives. The non-primary module identification is used to remove those packages without conducting pairwise comparisons, which improves efficiency. The non-obfuscated package identification is used to divide packages in the non-primary modules into obfuscated and non-obfuscated packages, which enables us apply the fast package name matching policy to non-obfuscated packages instead of applying the slow signature-based matching policy to all packages, to conduct pairwise comparisons. Thus, it is also helpful to improve efficiency. Further, we design filters to determine a perfectly matched package for a given ASP, which reduces false positives. Last, we propose a perfectly matched TPL determination approach by integrating local detection, online detection, and TPL discovery, which reduces false negatives. Our approach integrates these components to build a good solution to solve the TPL detection problem. Experiment results show that comparing with state-of-the-art approaches, LibRoad has a good performance and efficiency. To the best of our knowledge, this is the first work to not only detect TPLs with the accurate version but also provide the capacity of online detection and TPL discovery. Moreover, we propose two methods to build ground truth and provide benchmarks for colleagues to evaluate the performance of their TPL detection approaches.

Acknowledgments

The authors would like to thank the anonymous reviewers who have provided insightful and constructive comments that have led to important improvements in several parts of the manuscript. This work was supported in part by the National Natural Science Foundation of China under Grant numbers 61872186.

References

[1] W. Hu *et al.*, "Duet: Library integrity verification for Android applications," in *Proc. ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2014, pp. 141–152.
[2] A. Mylonas, M. Theoharidou, and D. Gritzalis, "Assessing privacy risks in Android: a user-centric approach," *Risk Assessment Risk-Driven Testing*, vol. 8418, pp. 21–37, 2014.
[3] J. L. Lin, B. Liu, N. Sadeh, and J. I. Hong, "Modeling users' mobile app privacy preferences: Restoring usability in a sea of permission settings," in *Proc. Symp. Usable Privacy Secur.*, 2014. pp. 199–212.
[4] H. Y. Wang, J. I. Hong, and Y. Guo, "Using text mining to infer the purpose of permission use in mobile apps," in *Proc. ACM Int. Joint Conf. Pervasive Ubiquitous Comput.*, 2015, pp. 1107–1118.
[5] X. Zheng, L. Pan, and E. Yilmaz, "Security analysis of modern mission critical Android mobile applications," in *Proc. Australas. Comput. Sci. Week MultiConf.*, 2017, Art. no. 2.
[6] K. Chen, P. Liu, and Y. Zhang, "Achieving accuracy and scalability simultaneously in detecting application clones on Android markets," in *Proc. 36th Int. Conf. Softw. Eng.*, 2014, pp. 175–186.
[7] Z. Wu *et al.*, "Fast, scalable detection of Piggybacked mobile applications," in *Proc. ACM Conf. Data Appl. Secur. Privacy*, 2013, pp. 185–196.
[8] W. Zhou, Y. J. Zhou, X. X. Jiang, and P. Ning, "Detecting repackaged smartphone applications in third-party Android marketplaces," in *Proc. 2nd ACM Conf. Data Appl. Secur. Privacy*, 2012, pp. 317–326.
[9] S. Hanna, L. Huang, E. Wu, S. Li, C. Chen, and D. Song, "Juxtapp: A scalable system for detecting code reuse among Android applications," in *Proc. 9th Conf. Detection Intrusions Malware Vulnerability Assessment*, 2012, pp. 62–81.
[10] H. Y. Wang, Z. Y. Wang, Y. Guo, and X. Q. Chen, "Detecting repackaged Android applications based on code clone detection technique," *Sci. China: Inf. Sci.*, vol. 44, no. 1, pp. 142–157, 2014.
[11] C. Gibler, R. Stevens, J. Crussell, H. Chen, H. Zang, and H. Choi, "AdRob: Examining the landscape and impact of Android application plagiarism," in *Proc. 11th Annu. Int. Conf. Mobile Syst. Appl. Serv.*, 2013, pp. 431–444.
[12] P. Pearce, A. P. Felt, G. Nunez, and D. Wagner, "Addroid: Privilege separation for applications and advertisers in Android," in *Proc. 7th ACM Symp. Inf. Comput. Commun. Secur.*, 2012, pp. 71–72.
[13] C. McMillan, M. Grechanik, and D. Poshyvanyk, "Detecting similar software applications," in *Proc. 34th Int. Conf. Softw Eng.*, 2012, pp. 364–374.

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.

[14] Y. Zhou and X. Jiang, "Dissecting Android malware: Characterization and evolution," in *Proc. IEEE Symp. Secur. Privacy*, 2012, pp. 95–109.
[15] S. Shekhar, M. Dietz, and D. S. Wallach, "AdSplit: Separating smartphone advertising from applications," in *Proc. 21st USENIX Conf. Secur. Symp.*, 2012, pp. 28–28.
[16] X. Zhang, A. Ahlawat, and W. Du, "AFrame: Isolating advertisements from mobile applications in Android," in *Proc. 29th Annu. Comput. Secur. Appl.*, 2013. pp. 9–18.
[17] L. Li *et al.*, "An investigation into the use of common libraries in Android apps," in *Proc. IEEE 23rd Int. Conf. Softw. Anal. Evol. Reengineering*, 2016, vol. 1, pp. 403–414.
[18] M. Grace, W. Zhou, X. Jiang, and A. Sadeghi, "Unsafe exposure analysis of mobile in-app advertisements," in *Proc. 5th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2012, pp. 101–112.
[19] T. Book, "Longitudinal analysis of Android ad library permissions," in *Proc. IEEE Mobile Secur. Technol.*, 2013, *arXiv:1303.0857*.
[20] J. L. Lin, S. Amini, J. I. Hong, N. Sadeh, J. Lindqvist, and J. Zhang, "Expectation and purpose: Understanding users' mental models of mobile app privacy through crowdsourcing," in *Proc. ACM Conf. Ubiquitous Comput.*, 2012, pp. 501–510.
[21] H. Y. Wang, Y. Guo, Z. A. Ma, and X. Q. Chen, "WuKong: A scalable and accurate two-phase approach to Android app clone detection," in *Proc. ACM Int. Symp. Softw. Testing Anal.*, 2015. pp. 71–82.
[22] A. Narayanan, L. Chen, and C. K. Chan, "Addetect: Automated detection of Android ad libraries using semantic analysis," in *Proc. 9th Int. Conf. Intell. Sens. Sensor Netw. Inf. Process.*, 2014, pp. 1–6.
[23] B. Liu *et al.*, "Efficient privilege de-escalation for ad libraries in mobile apps," in *Proc. 13th Annu. Int. Conf. Mobile Syst. Appl. Serv.*, 2015.pp. 89–103.
[24] J. Crussell, C. Gibler, and H. Chen, "AnDarwin: Scalable detection of semantically similar Android applications," in *Proc. 18th Eur. Symp. Res. Comput. Secur.*, 2013, pp. 182–199.
[25] M. Li *et al.*, "LibD: Scalable and precise third-party library detection in Android markets," in *Proc. 39th Int. Conf. Softw. Eng.*, 2017, pp. 335–346.
[26] M. Backes, S. Bugiel, and E. Derr, "Reliable third-party library detection in Android and its security applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 356–367.
[27] Y. Zhang *et al.*, "Detecting third-party libraries in Android applications with high precision and recall," in *Proc. 25th Int. Conf. Softw. Anal. Evol. Reengineering*, 2018. pp. 141–152.
[28] Apktool, Jun. 2020. [Online]. Available: http://ibotpeaches.github.io/Apktool/.
[29] Z. Ma, H. Wang, Y. Guo, and X. Chen, "Libradar: Fast and accurate detection of third-party libraries in Android apps," in *Proc. 38th Int. Conf. Softw. Eng. Companion*, 2016, pp. 653–656.
[30] Maven central repository, Jun. 2020. [Online]. Available: https://search.maven.org/
[31] Dexguard, Jun. 2020. [Online]. Available: https://www.guardsquare.com/en/dexguard
[32] Z. Tang *et al.*, "Securing Android applications via edge assistant third-party library detection," *Comput. Secur.*, vol. 80, pp. 257–272, 2019.
[33] P. Wang *et al.*, "Software protection on the go: A large-scale empirical study on mobile app obfuscation," in *Proc. ACM/IEEE 40th Int. Conf. Softw. Eng.*, 2018, pp. 26–36.
[34] P. Graux, J. F. Lalande, and V. V. T. Tong., "Obfuscated Android application develop," in *Proc. 3rd Central Eur. Cybersecurity Conf.*, 2019, pp. 1–6.



Jian Xu received the PhD degree in computer science, from the Nanjing University of Science and Technology, Nanjing, China, in 2007. Currently, he holds the position of a professor at the Nanjing University of Science and Technology. His research interests include software analysis and data mining in which he has published more than 40 papers in journals and refereed conference proceedings.



Qianting Yuan is currently working toward the master's degree at the Nanjing University of Science and Technology, Nanjing, China. Her major research interest includes third-party library detection.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: University Bern. Downloaded on January 19,2026 at 19:26:21 UTC from IEEE Xplore. Restrictions apply.